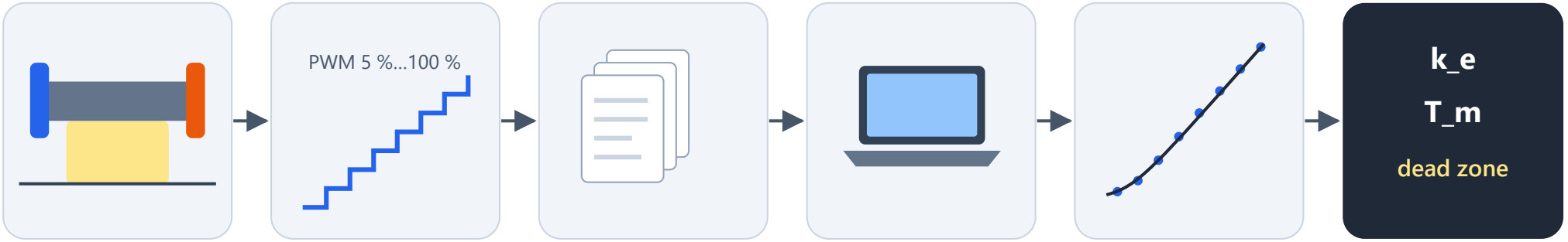
Motor parameter identification

80 automated step tests → a model of each drive motor

The whole thing in one picture



1 · Lift robot
wheels spin free

2 · Run program
40 levels × 2 wheels

3 · 80 data files
time · encoder angle

4 · Copy to laptop
pull_data.py (scp)

5 · Fit model
analyze.py (curve_fit)

6 · Parameters
→ Week 6 odometry

Question
How does each wheel motor respond when we apply a voltage? How fast, and how quickly does it get there?

Method
Apply a voltage step, record the encoder angle, fit a first-order model. Repeat at every PWM level, for both wheels.

Why it matters
The numbers feed later labs: predicting robot motion (odometry) and tuning speed control.

Result at a glance (robot lifted · wheel 1 M4 / wheel 2 M3)

k_e [V·s/rad]
1.03 / 1.22

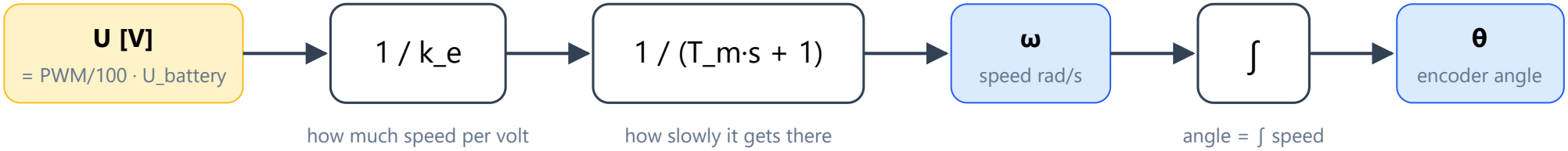
T_m (median)
145 / 172 ms

top speed at 100 %
16.7 / 16.4 rad/s

dead zone
none
on the floor: 10 to 15 %

1 The motor model

what we are trying to measure



how much speed per volt

how slowly it gets there

angle = ∫ speed

$$T_m \cdot d\omega/dt + \omega = U / k_e$$

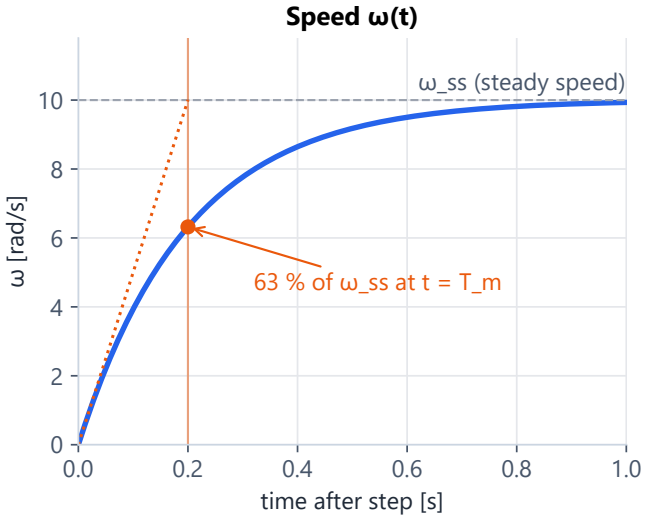
first-order system: speed approaches its target exponentially

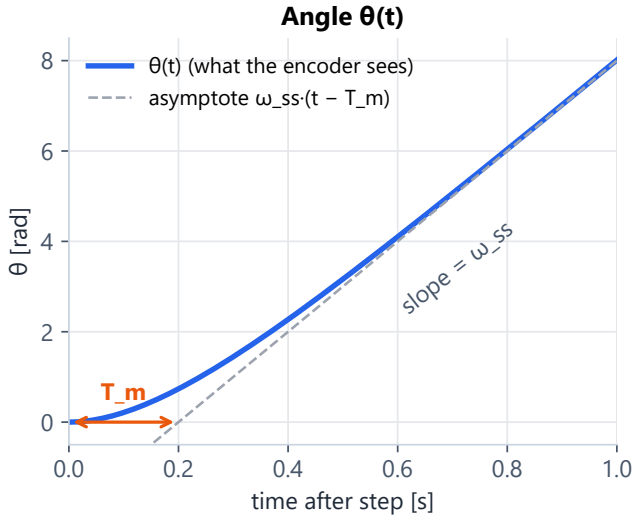
ω_ss steady speed after the start-up. In the model ω_ss = U / k_e.

k_e [V·s/rad] volts needed per rad/s of speed (back-EMF constant). Bigger k_e = slower wheel.

T_m [s] time constant. After T_m the wheel is at **63 %** of ω_ss, after 3·T_m at 95 %.

Dead zone smallest PWM that turns the wheel. Below it, friction wins.



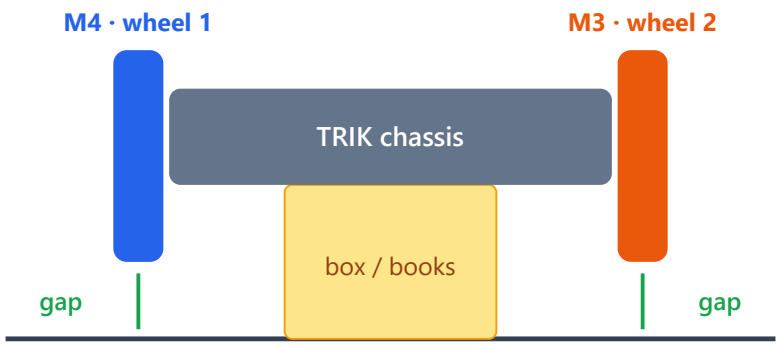


Key trick: the encoder gives **angle**, not speed. So we fit the integrated curve $\theta(t) = \omega_ss \cdot (t - T_m + T_m \cdot e^{-t/T_m})$. Late in the run it becomes a straight line: its **slope is ω_ss**, and it is **shifted right by T_m**.

2 Experiment design

one step test, repeated 80 times

Setup: robot lifted



front view · wheels touch nothing

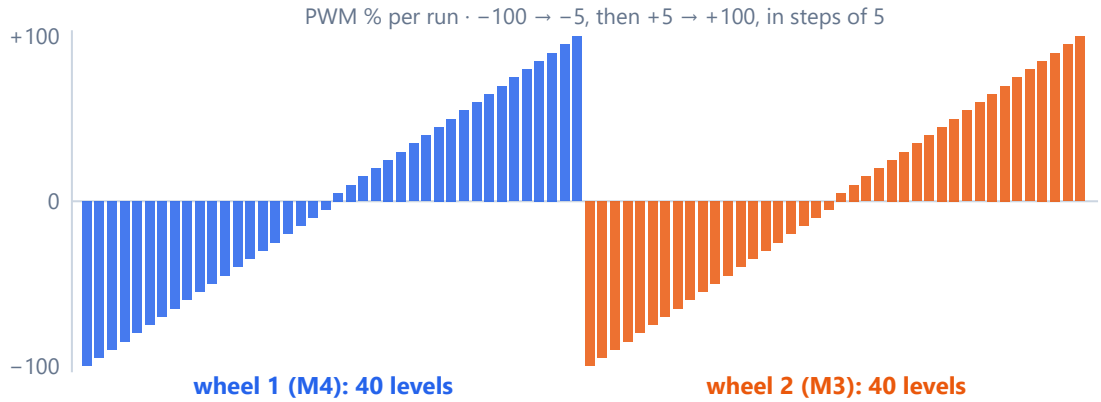
Why lift? On the floor the wheels also push the robot's mass and fight tyre friction. Lifted, we measure the motor alone (page 7 compares both).

Voltage, not just PWM

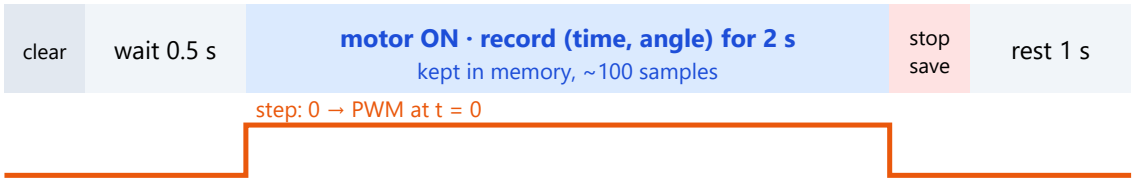
$$U = PWM / 100 \cdot U_battery$$

The battery is read **during** each run (under load), since it sags as the motor draws current: 12.36 V idle → 12.15 V at the end.

The sweep: 80 runs, about 5 minutes



One run (3.5 s)

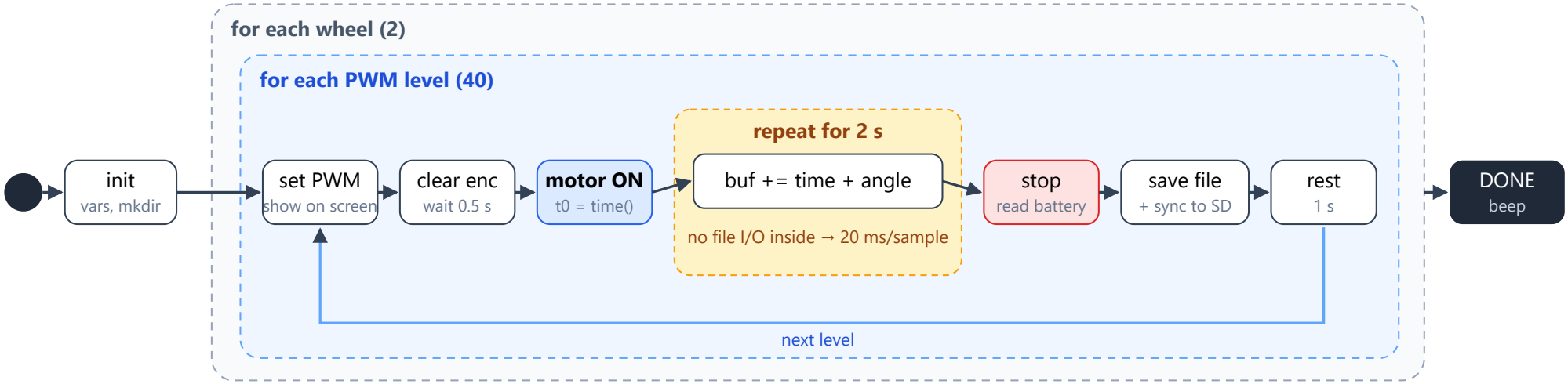


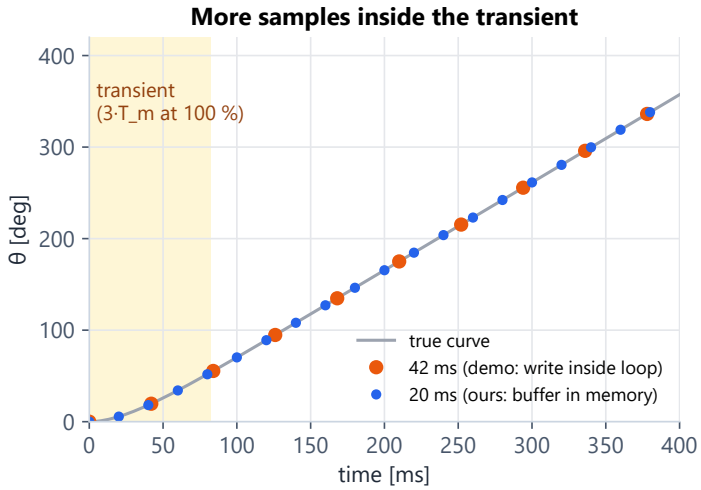
Clear encoder so every file starts at θ = 0.

Rest 1 s so the wheel has stopped before the next step: the model assumes a start from rest.

3 Automation

motor_identification_diagram.qrs: one upload, no clicking per run





Blocks can't create folders, read the battery or name files from variables, so 4 "System call" blocks hold short JavaScript snippets.

Sampling speed matters. The demo writes to the file inside the loop: one sample every 42 ms. At high PWM the whole start-up (3·T_m ≈ 100 ms) gets only 2 to 3 points, too few to fit T_m. Buffering in memory gives 20 ms.

Data safety. Files are written to the SD card with sync right after each run. Without it, a power-off lost all wheel 2 files (0 bytes) in an earlier run.

On the laptop, 2 commands:

```
# copy dilyorm_motor_data/ from the robot over Wi-Fi
python pull_data.py --host 192.168.50.254
# fit all 80 files → report.md + figures/
python analyze.py
```

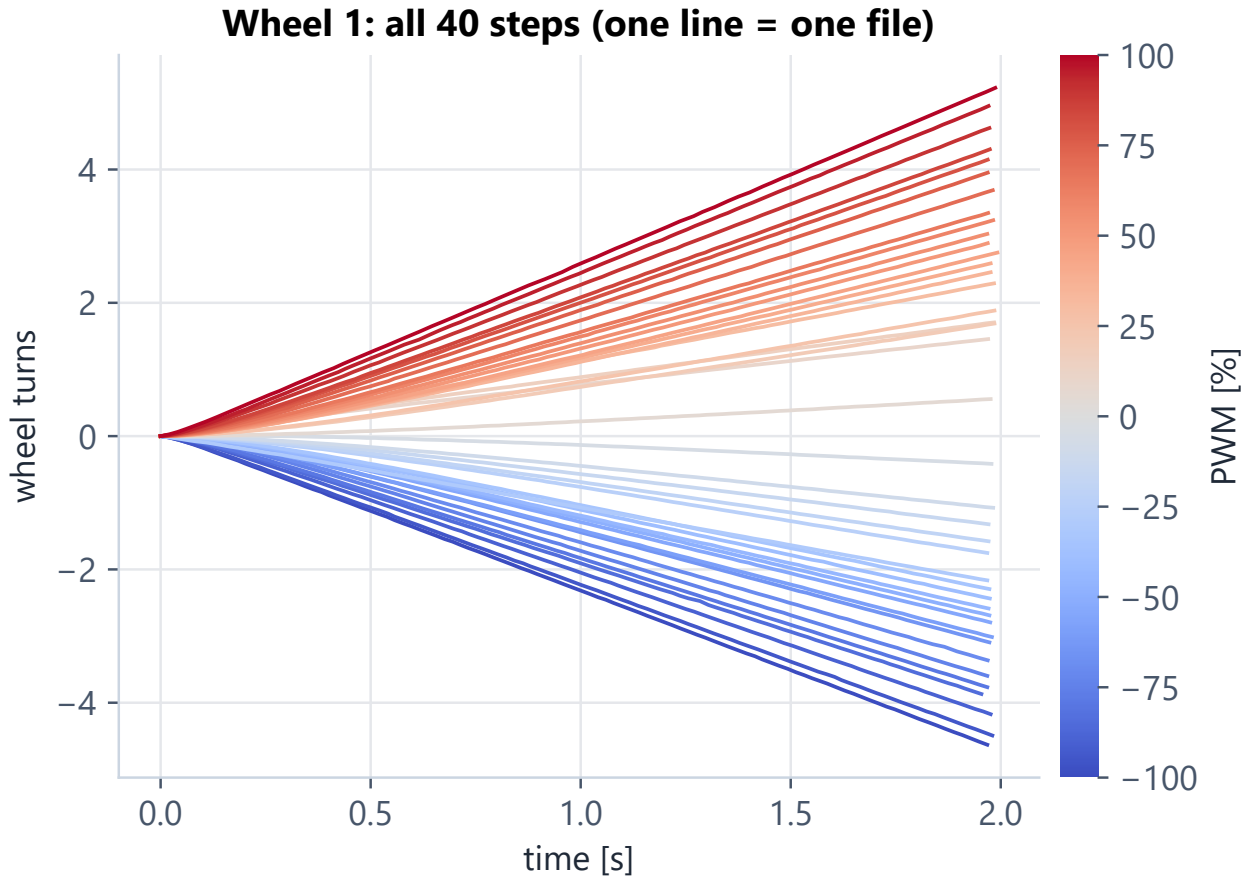
4 The raw data

80 files: wheel_<N>_v_<PWM>.txt

One file

```
# wheel_id: 1
# wheel_label: left
# motor_port: M4
# encoder_port: E4
# pwm_percent: 50
# battery_voltage: 12.17
# battery_voltage_loaded: 12.17
# format: time_ms angle_deg
34 0
54 3
74 8
95 15
…
1984 1029
2003 1042
```

Header holds everything needed to compute U, so each file can be analysed on its own. The **body** is time in ms and wheel angle in degrees.

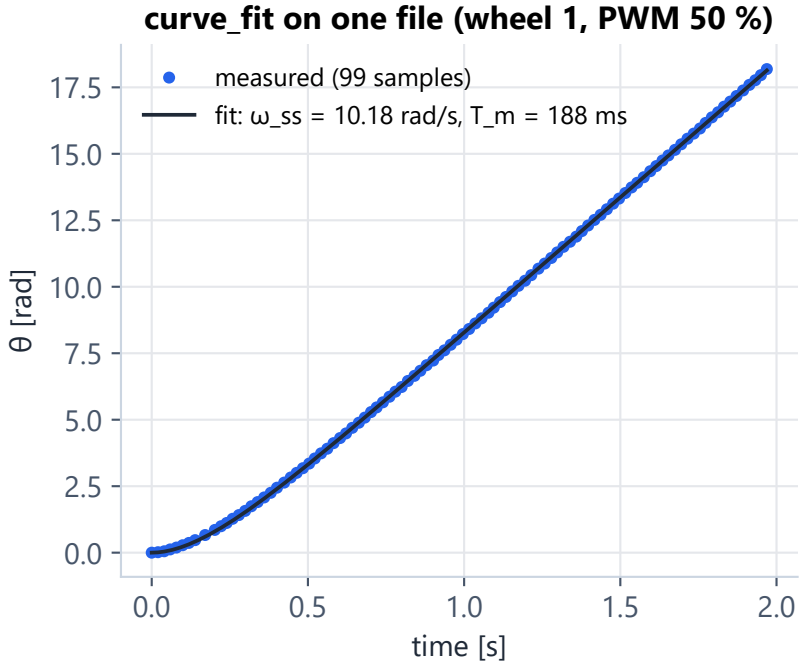


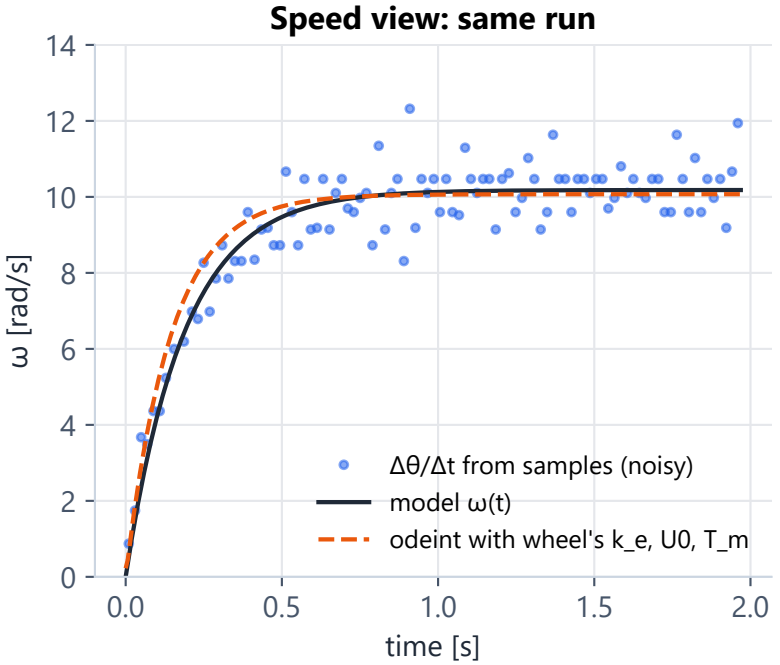
Each line is one step test. It curves at first (accelerating), then goes straight (constant speed).

Red up, blue down: sign of PWM = direction. Steeper line = higher speed, so the fan shows speed vs PWM.

5 Fitting one run

scipy curve_fit: 2 unknowns per file





$$\theta(t) = \omega_ss \cdot (t - T_m + T_m \cdot e^{-t/T_m})$$

find ω_ss and T_m that minimise the squared error

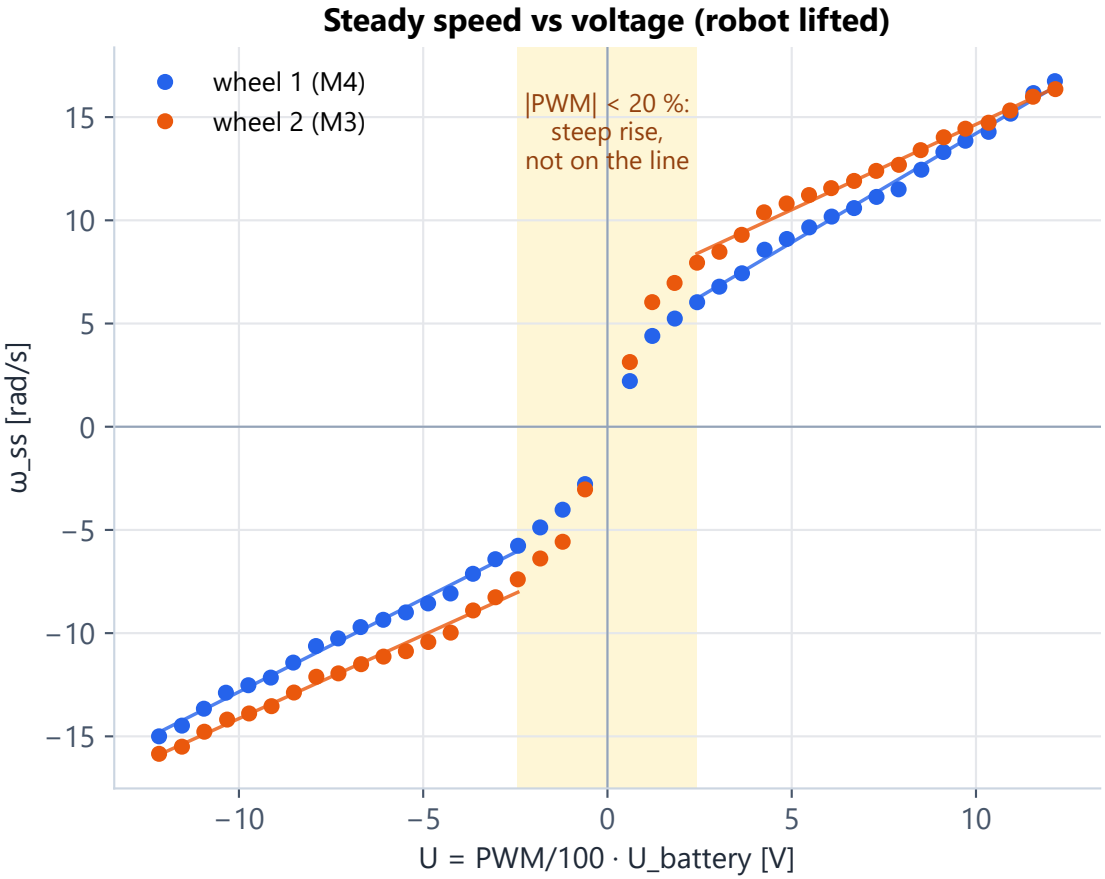
Why fit angle, not speed? Speed from differences (right plot, dots) is noisy: 1° of encoder over 20 ms = 0.9 rad/s. Angle is clean (left plot).

Trust check: T_m only counts if ≥ 4 samples fall inside the start-up (t < 3·T_m). Otherwise it is marked "unreliable".

Orange dashed = `odeint` simulation with the wheel's final k_e, U0, T_m. It lands close to the measurement, so the model works.

6 From 80 fits to k_e

plot every ω_ss against its voltage



$$\omega_ss = (U - U0) \,/\, k_e$$

straight line fitted on |PWM| ≥ 20 %, each direction separately

Slope = 1/k_e. Steeper line = more speed per volt = smaller k_e.

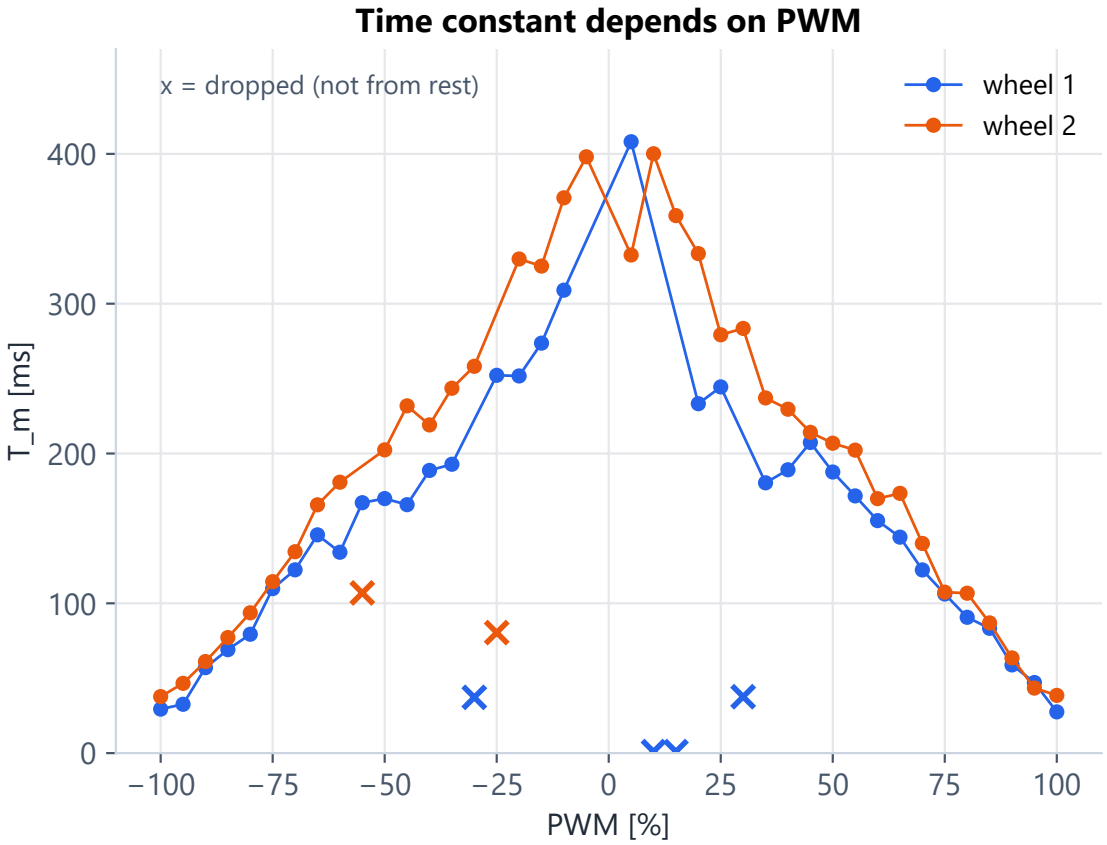
Not quite a line. Speed shoots up to ≈ 5 rad/s within the first 10 % PWM, then grows slowly. So the fitted line does not pass through 0: extended to 0 V it would still give ≈ 3.6 rad/s (that's why U0 < 0 forward). Use the line for |PWM| ≥ 20 % and a lookup table below that.

	K_E FWD	K_E REV	Ω @100 %
wheel 1	0.95	1.11	16.7
wheel 2	1.21	1.24	16.4

k_e in V·s/rad, ω in rad/s

7 Time constant T_m

how quickly the wheel reaches speed



145 / 172 ms

median T_m, wheel 1 / wheel 2 (|PWM| ≥ 20 %)

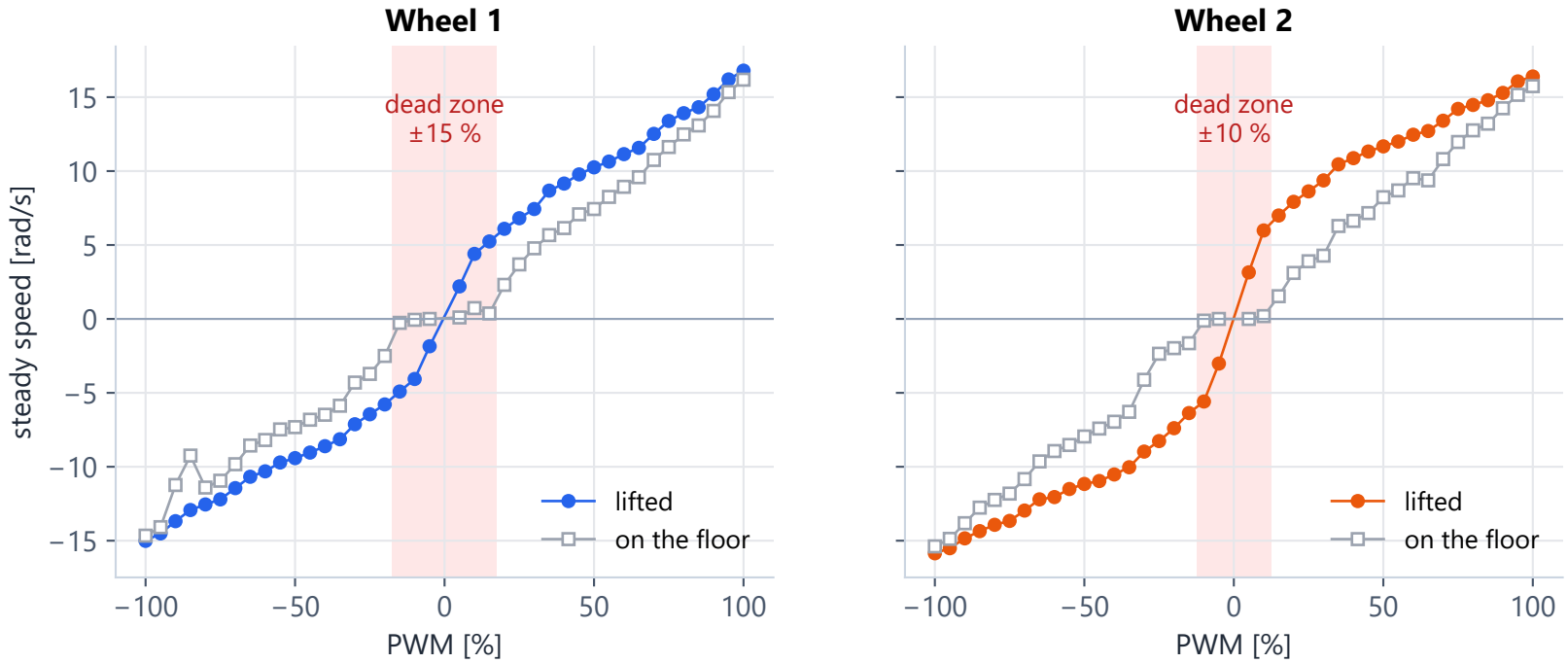
Ideal motor: T_m is a constant, depending only on inertia and the motor.

Ours: ≈ 400 ms at 5 to 10 % falls to ≈ 30 to 40 ms at 100 %. Friction and the PWM driver make the motor look sluggish at low voltage. One T_m is a compromise; the median is used.

Practical meaning: after a speed command, expect about 0.5 s (3·T_m at mid PWM) before the wheel is at speed. A controller faster than that will fight the motor.

8 Floor vs lifted

same program, run twice



	DEAD ZONE	K_E	T_M	Ω @100 %
wheel 1 floor → lifted	±15 % → **none**	0.84 → 1.03	245 → 145 ms	16.2 → 16.7 rad/s
wheel 2 floor → lifted	±10 % → **none**	0.80 → 1.22	235 → 172 ms	15.7 → 16.4 rad/s

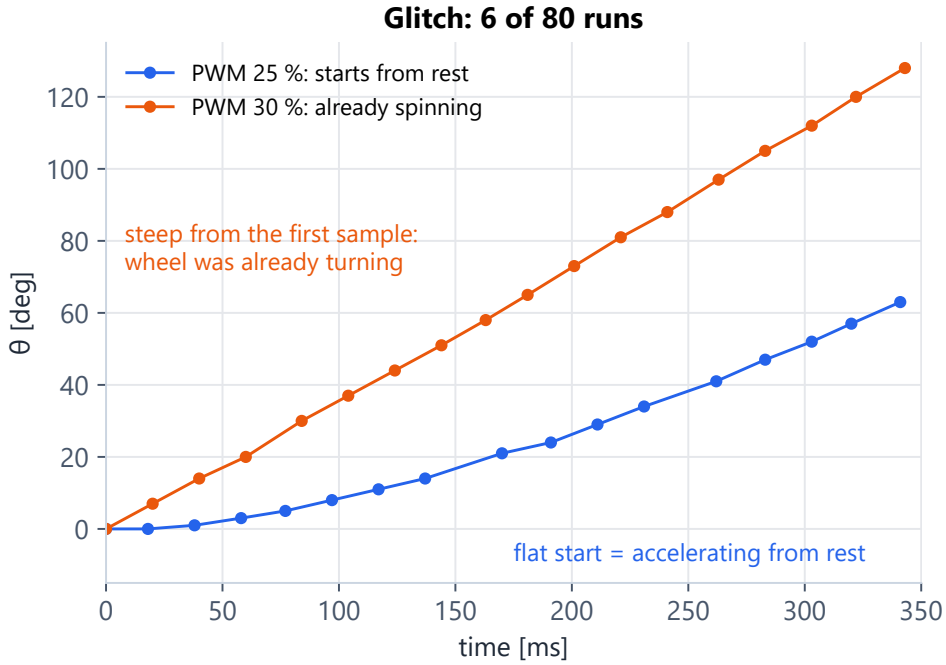
Dead zone comes from the load. Lifted, the wheels turn already at 5 %. On the floor they need 10 to 15 % to overcome tyre and robot friction.

Lifted is faster to respond: T_m drops by about 40 % with no robot mass to accelerate.

k_e alone misleads: it is a slope on |PWM| ≥ 20 %, and the floor curve is shifted and straighter. Compare whole curves, not one number.

9 Checking the data

what went wrong, and how the pipeline handles it



Wheel already turning at t = 0 in 6 of 80 runs (also in 7 runs of the professor's reference data, so it is the robot, not our setup). The fit then gives a wrong T_m (as low as 1 ms). `analyze.py` detects these runs (start much faster than the neighbouring levels), **drops their T_m** and takes ω_ss from the straight end part only.

Empty files after power-off

Linux caches writes in memory. Fix: `sync` after each file (diagram) / `os.fsync` (Python); the analyzer skips files with < 5 rows.

Battery near cut-off

12.15 V under load at the end; robot low-power limit is 12 V. Low voltage = lower top speed. Charge before each series; U is computed from the measured voltage anyway.

Timing jitter

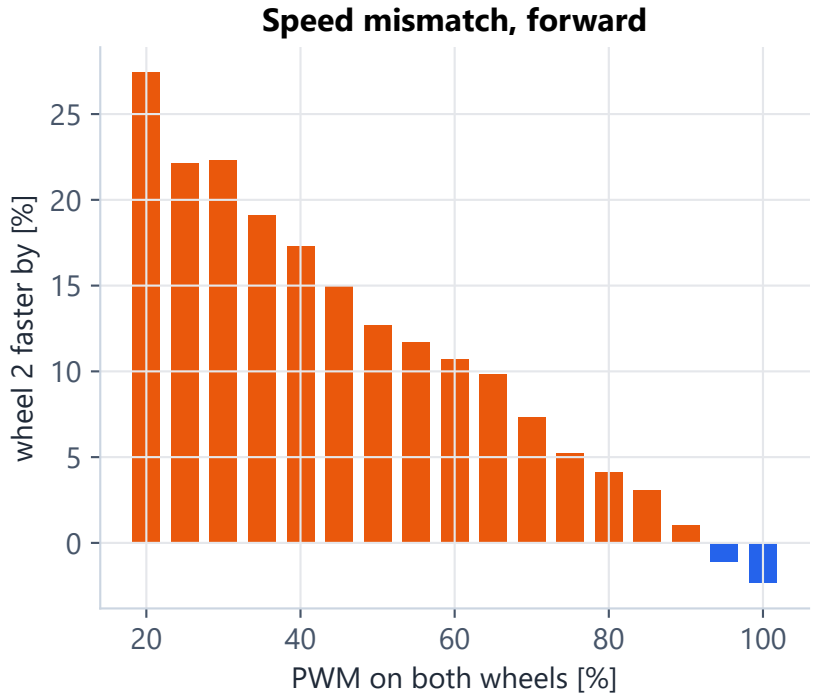
Median 20 ms, worst gaps 30 to 60 ms. Real timestamps are stored, so the fit uses true times, not an assumed period.

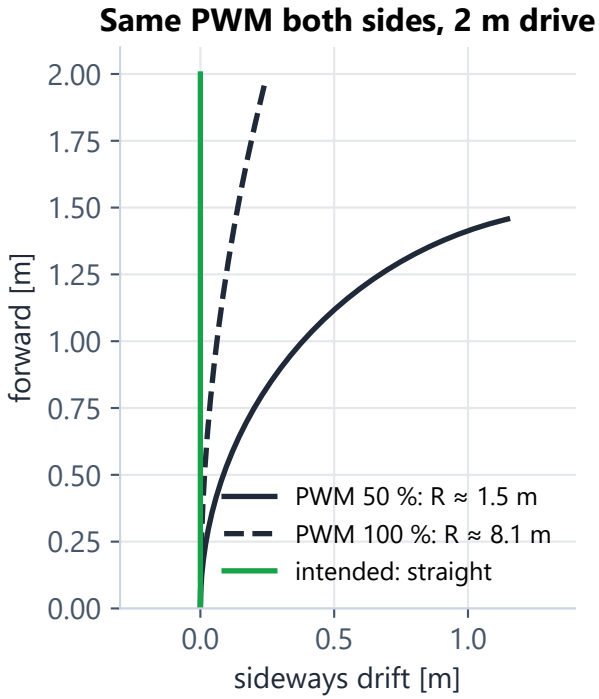
Encoder direction

If an encoder counts backwards vs. its motor, the analyzer detects the sign and reports it. Raw files are never edited.

10 Two wheels are not twins

why the robot won't drive straight on equal PWM





$$R \approx \text{track} / \varepsilon, \quad \varepsilon = (\omega_2 - \omega_1) / \omega_\text{mean}$$

track = 0.19 m between the wheels

At 50 %: wheel 2 is 12.7 % faster → circle of R ≈ 1.5 m (≈ 38° of heading per metre). **At 100 %:** only 2.3 % apart (wheel 1 slightly faster, blue bars) → R ≈ 8 m.

For Week 6: the mismatch changes with speed, so one fixed correction factor won't work. Use per-wheel models, or better, close the loop on the encoders.

Summary

robot lifted · 80 runs · 15 Oct 2026

PARAMETER	WHEEL 1 · M4	WHEEL 2 · M3
k_e [V·s/rad] (fwd / rev)	**1.03** (0.95 / 1.11)	**1.22** (1.21 / 1.24)
U0 line offset [V] (fwd / rev)	−3.45 / +4.25	−7.71 / +7.48
T_m median [ms]	**145**	**172**
ω_ss at 100 % [rad/s]	**16.7**	**16.4**
dead zone lifted / floor	none / ±15 %	none / ±10 %
battery (loaded) [V]	12.17	12.15
sample period [ms]	20	20

Files: `motor_identification_diagram.qrs` (robot program) · `motor_data/` (80 raw files) · `analyze.py` → `report.md`, `figures/`

Five takeaways

1 · A first-order model fits each step well.

2 · But the motor isn't linear: speed vs U is curved and T_m shrinks with PWM.

3 · The dead zone is load: gone when lifted, 10 to 15 % on the floor.

4 · The wheels differ, and by an amount that depends on speed, so equal PWM drives an arc.

5 · Automate and check: 80 runs in 5 minutes, with glitches flagged automatically.